

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C programlama dili, yapısal olmasının yanında aynı zamanda modüler bir programlama dilidir. **Modüler programlamada** (**modular programming**) yazılan fonksiyonlar gruplar halinde başka kod dosyalarına konulur ve bu bileşenler gerektiğinde projemize **#include ön işlemci yönergeleriyle** (**preprocessor directive**) dahil edilir.

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağrılması, işlemciyi yorar. Çünkü her fonksiyon çağrısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C programlama dilinde en çok kullanacağımız modüllerden biri de standart giriş-çıkış yani **IO** (**input-output**) işlemlerinin tanımlı olduğu **stdio.h** başlık (**header**) dosyasıdır.

Bilgi

Bir modülü kodumuza **dahil etmek** (**include**) için kaynak kodun başında **#include** ön işlemci yönergesi (**preprocessor directive**) kullanılır.

Kodun başına **#include <stdio.h>** ekleyerek **konsola** (konsol kelimesi UNIX/Linux işletim sistemindeki terminal, Windows işletim sisteminde ise komut satırını ifade eder) bir şey yazmak için **printf()** veya klavyeden bir şey okumak ve bir değişkene atamak için ise **scanf()** fonksiyonlarını kullanabilir hale geliriz.

C programlama dilinde her işletim sisteminde de çalışan standart birçok başlık dosyası bulunmaktadır. Ancak bunların dışında, işletim sistemine özel olan başlık dosyaları da bulunmaktadır;

Başlık Dosyası	Açıklama
stdio.h	Standart giriş-çıkış komutları
math.h	Matematiksel fonksiyonlar
stdlib.h	Dönüşüm, sıralama, arama vb. komutları
string.h	Alfa sayısal ve bazı bellek yönetim komutları
curses.h	Curses, metin terminali ekranında imleç (cursor) denetimini destekleyen eski bir Unix/Linux kütüphanesidir.
conio.h	DOS ve Windows komut satırında imleç (cursor) denetimini destekleyen kütüphanedir

Tablo 5.1: En Çok Kullanılan Başlık Dosyaları

5.2 Konsola Biçimlendirilmiş Veri Yazma

C programlama dilinde değişkenlerimizde olan veriyi **biçimlendirilmiş** (**formatted**) bir şekilde imlecin bulunduğu noktadan itibaren konsola yazabiliriz (**print**). Bunu sağlayan fonksiyon **stdio.h** başlık dosyasındaki **printf()** fonksiyonudur.

Bir fonksiyonun hangi parametreleri aldığı ve neyi geri döndürdüğünü **prototipinden** (**prototype**) anlarız. Aşağıda **printf()** fonksiyonunun prototipi verilmiştir;

```
int printf (const char *format, ... );
```

Bu fonksiyon bir **biçimlendirme metnini** (**format string**) ilk argüman olarak alır ve konsola yazdırılan karakter sayısını döndürür.

Bilgi

C dilinde **metinler**, **çift tırnak karakterleri arasında** yazılır!
Örnek olarak "İlhan", "Merhaba! Mehmet" verilebilir.

Biçimlendirme metni, aynı zamanda kendisinden sonra gelecek argümanların hangi biçimde konsola yazılacağını belirler. Biçimlendirme metni, içinde kendisinden sonra gelen argümanların hangi biçimde yazılacağını belirten **biçim belirleyicisi** (**format specifier**) karakterleri içerir.

Bilgi

Biçim belirleyicileri iki karakter olup ilk karakteri yüzde (%) karakteridir.

Aşağıda konsola biçimlendirilmemiş ve biçimlendirilmiş metin yazan kod örneği verilmiştir;

```
1 /* Bu program, printf örneğidir. */
2 #include <stdio.h> //Konsola bir şey yazmak için gereklidir!
3 int main() {
4     int yas=50;
5     float kilo=100.0;
6     printf("Merhaba!");
7     printf("Yaşınız:%d",yas);
8     printf("Kilonuz:%f",kilo);
9     return 0;
10 }
11 /*Program Çıktısı:
12 Merhaba!Yaşınız:50Kilonuz:100.000000
13 */
```

Bu kodu aşağıdaki şekilde açıklayabiliriz;

- ◊ İkinci satırdaki yönerge ile **stdio.h** kütüphanesi kaynak kodumuza dahil edilir.
- ◊ Dördüncü ve beşinci satırdaki talimatlarla değişkenler başlatılır.
- ◊ Altıncı satırdaki **printf** fonksiyonu, sadece değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - İçinde biçim belirleyicisi yoktur!
 - Bu nedenle konsolda imlecin bulunduğu yerden itibaren "Merhaba!" yazar ve imleci (**cursor**) metnin sonuna taşır.
 - **Bu metinden sonra argüman olsa bile konsola bir şey yazılmaz!**
- ◊ Yedinci satırdaki **printf** fonksiyonu, içinde biçim belirleyicisi (%d) olan bir değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - Dolayısıyla **bu metinden sonra ondalık (decimal) olarak konsola yazdırılabilecek bir argüman verilmelidir.**
 - İkinci argüman olarak **yas** değişkeni verilmiştir.
 - Biçimlendirme metnindeki **%d** biçim belirleyicisi yerine **yas** değişkeninin değerini ondalık (**decimal**) olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin **"Yaşınız:50"** olacaktır.
 - İmlecin bulunduğu yerden itibaren **"Yaşınız:50"** yazar ve imleci (**cursor**) metnin sonuna taşır.
- ◊ Sekizinci satırdaki **printf** fonksiyonu, içinde biçim belirleyicisi (%f) olan bir değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - Dolayısıyla **bu metinden sonra kayan noktalı sayı olarak konsola yazdırılabilecek bir argüman verilmelidir.**
 - İkinci argüman olarak **kilo** değişkeni verilmiştir.
 - Biçimlendirme metnindeki **%f** biçim belirleyicisi yerine **kilo** değişkeninin değerini kayan noktalı (**floating point**) sayı olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin **"Kilonuz:100.000000"** olacaktır.
 - İmlecin bulunduğu yerden itibaren **"Kilonuz:100.000000"** yazar ve imleci (**cursor**) metnin sonuna taşır.

Biçim belirleyicileri sadece ondalık ve kayan noktalı sayılarla sınırlı değildir. Aşağıda listesi verilmiştir;

Eğer konsola % karakteri yazmak istersek biçimlendirme olarak iki yüzde (%) karakteri kullanmalıyız. Buna ilişkin örnek aşağıda verilmiştir;

```
/* Bu program, printf ile konsola % karakteri basma örneğidir. */
#include <stdio.h> //printf için gerekli!
```

Biçim Karakterleri	Argümanın Konsola Yazılma Biçimi
%d	Konsola onluk tabanda (decimal) sayı olarak yazılır.
%c	Konsola karakter olarak yazılır.
%f	Konsola kayan noktalı sayı (float) olarak yazılır.
%x	Konsola onaltılı tabanda (hexadecimal) sayı olarak yazılır.
%s	Konsola metin (string) olarak yazılır.
%%	Konsola % karakteri yazılır.

Tablo 5.2: Biçim Belirleyiciler

```
int main() {
    int doluluk=80;
    printf("Doluluk Oranı: %% %d",doluluk);
    return 0;
}
/*Program Çıktısı:
Doluluk Oranı: % 80
*/
```

Şu ana kadar tek bir argümanı biçimlendirilmiş olarak konsola yazmayı öğrendik. Birden çok argümanı da konsola yazdırabiliriz. **Bunun için biçimlendirme metni içerisinde izleyen argümanları sırasıyla biçimlendirecek biçim belirleyicileri kullanmalıyız.** Buna aşağıdaki örnek verilebilir.

```
1 /* Bu program, çok argümanlı printf örneğidir. */
2 #include <stdio.h>
3 int main() {
4     char yas=45;
5     float kilo=81.0;
6     int doluluk=80;
7     printf("Yaş:%d, Kilo:%f, Doluluk Oranı:%% %d",yas,kilo,doluluk);
8     return 0;
9 }
10 /* Program Çıktısı:
11 Yaş:45, Kilo:81.000000, Doluluk Oranı:% 80
12 */
```

Bu kodu aşağıdaki şekilde açıklayabiliriz;

- ◊ İkinci satırdaki yönerge ile `stdio.h` kütüphanesi kaynak kodumuza dahil edilir.
- ◊ Dördüncü, beşinci ve altıncı satırdaki talimatlarla değişkenler başlatılır.
- ◊ Yedinci satırdaki `printf` fonksiyonu, içinde sırasıyla `%d`, `%f`, `%%` ve `%d` biçim belirleyicileri olan bir değişmez metin (**string literal**) argümanı ile çağrılmıştır. Dolayısıyla bu metinden sonra **sırasıyla konsola yazdırılabilecek bir onluk tabanda sayı, bir kayan noktalı sayı ve yine bir onluk tabanda sayı** argüman olarak verilmelidir. Yüzde karakteri (`%%`) için argüman aranmaz. Konsola yazdırılacak metin aşağıdaki şekilde oluşturulur;
 - Biçimlendirme metnindeki ilk `%d` biçim belirleyicisi yerine ilk argüman olan `yas` değişkeninin değerini onluk tabanda sayı (**decimal**) olarak konulur.
 - Biçimlendirme metnindeki ikinci `%f` biçim belirleyicisi yerine ikinci argüman olan `kilo` değişkeninin değerini kayan noktalı sayı (**float**) olarak konulur.
 - Biçimlendirme metnindeki üçüncü `%%` biçim belirleyicisi yerine sadece `%` karakteri konulur.
 - Biçimlendirme metnindeki son `%d` biçim belirleyicisi yerine son argüman olan `doluluk` değişkeninin değerini onluk tabanda sayı (**decimal**) olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin `"Yaş:45, Kilo:81.000000, Doluluk Oranı:% 80"` olacaktır.
 - İmlecin bulunduğu yerden itibaren nihai hali almış olan biçimlendirilmiş metin konsola yazılır ve imleç (**cursor**) metnin sonuna taşınır.

Biçimlendirme metninde imleci hareket ettirecek şekilde karakterler kullanabiliriz. Biçimlendirme metni içine imlecin satır başına geçmesini sağlamak için `\n` şeklinde bir dizi karakter kullanarak imleci kaçırabiliriz. Bu tür karakter dizilerine **kaçış dizisi (escape sequence)** adı verilir. Bunlar aşağıda verilmiştir;

Aşağıda kaçış dizilerine ilişkin örnek program verilmiştir.

```
/* Bu program, escape sequence printf örneğidir. */
#include <stdio.h>
int main() {
```

Kaçış Dizisi	Açıklama
<code>\n</code>	Konsolda imleç yeni satırın (new line) başına hareket eder.
<code>\t</code>	İmlece, metin yazarken sekme (tab) tuşuna basında olan hareketi yapar. Yani imleç, konsolda sekizin katları olan sütuna hareket eder.
<code>\r</code>	Konsolda imleç bulunduğu satırın başına (carriage return) hareket eder.
<code>\v</code>	Konsolda imleç bulunduğu yerden alt satıra dikey olarak (vertical) hareket eder.
<code>\a</code>	Konsol, uyarı (alert) sesi çıkarır.
<code>\\</code>	Konsola ters bölü karakterini yazar.
<code>\"</code>	Konsola çift tırnak karakterini yazar.
<code>'</code>	Konsola tek tırnak karakterini yazar.

Tablo 5.3: Kaçış Dizileri

```
printf("ILHAN\n"); // ILHAN yazılarak konsolda satır başı yapılır.
printf("OZKAN\r"); // OZKAN yazılarak aynı satırın başına dönülür.

printf("ABC\tDEF\n");
/* ABC yazılır ve 8. sütuna ilerlenir ve sonra DEF yazılır, sonrasında satır başı
yapılır. TAB karakteri 8 ve katları olan sütunlara imleci ilerletir.*/

printf("\"İyi\"-\"Kötü\""); // Çift tırnak içeren "İyi"- "Kötü" yazılır.
return 0;
}
/* Program Çıktısı:
ILHAN
ABCAN DEF
"İyi"- "Kötü"
*/
```

Biçim belirleyiciler (**format specifier**) ile aşağıdaki desen kullanılarak daha fazla biçimlendirme yapılabilir:

`[%[önek][genişlik][.ondalık]<biçim karakteri>`

Burada;

1. **Genişlik**, konsola kaç karakter genişliğinde yazdırılacağını belirler.
2. **Ondalık**, konsola kayan noktalı sayı yazılacak ise noktadan sonra kaç rakam yazdırılacağını belirler.
3. **Önek** içerisinde aşağıdaki karakterler olabilir;
 - (a) **+** karakteri: Sayının işaretini konsola yazılmasını sağlar.
 - (b) **-** karakteri: Yazdırılacak parametrenin belirtilen genişliğin soluna yaslanarak yazdırılmasını sağlar.
 - (c) **0** karakteri: Yazdırılacak sayının soluna belirtilen genişliğe bağlı olarak **0** karakteri konulmasını sağlar.

Aşağıda 10 karakterlik genişlikte **PI** sayısının biçimlendirme örnekleri verilmiştir;

```
/* Bu program, detaylı biçimlendirme printf örneğidir. */
#include <stdio.h>
int main() {
    const float PI=3.141527;
    /*Büyük karakteri biçimlendirmenin nerede başladığını belirtir.*/
    /*Küçük karakteri biçimlendirmenin nerede bittiğini belirtir.*/
    printf(">1234567890<\n"); //Çıktı: >1234567890<
    printf(">%.2f<\n",PI); //Çıktı: >3.14<
    printf(">%3.f<\n",PI); //Çıktı: > 3<
    printf(">%10.2f<\n",PI); //Çıktı: > 3.14<
    printf(">+10.3f<\n",PI); //Çıktı: > +3.142<
    printf(">+10.4f<\n",PI); //Çıktı: > +3.1415<
    printf(">%010.4f<\n",PI); //Çıktı: >00003.1415<
    printf(">-10.4f<\n",PI); //Çıktı: >3.1415 <
    printf(">+10.2f<\n",PI); //Çıktı: >+3.14 <
    printf(">+10.2f<\n",-PI); //Çıktı: >-3.14 <
    printf(">+10.2f<\n",-PI); //Çıktı: > -3.14<
    printf(">%0+10.2f<\n",-PI); //Çıktı: >-000003.14<
```

```
    return 0;
}
```

5.3 Klavyeden Biçimlendirilmiş Veri Okuma

C programlama dilinde değişkenlerimizde klavyeden biçimlendirilmiş (**formatted**) olarak girilen metin satırı tarayıp (**scan**), veriyi tutacak değişkenlere koyabiliriz. Bunu sağlayan fonksiyon **stdio.h** başlık dosyasındaki **scanf** fonksiyonudur. Bu fonksiyonunun prototipi aşağıda verilmiştir;

```
int scanf (const char *format, ... );
```

Bu fonksiyon da bir metni ilk olarak argüman olarak alır. İlk argüman olan bu **biçimlendirme metni** (**format string**) aynı zamanda kendisinden sonra gelecek argümanların da hangi biçimde klavyeden okunacağını belirler. Argümanların hangi biçimde okunacağını **printf()** fonksiyonu gibi **biçim belirleyicisi** (**format specifier**) belirler. Sonuç olarak okunan değişken sayısını döndürür.

Bilgi

scanf() Fonksiyonunda **printf()** fonksiyonundan farklı olarak, okunan değerlerin konulacağı **değişkenlerin adresleri argüman olarak verilir**. Bir değişkenin adresi, **&** işleci ile elde edilir.

Aşağıda kapasite oranını okuyup % olarak ekrana yazan bir program örneği verilmiştir.

```
/* Bu program, detaylı biçimlendirme scanf örneğidir. */
#include <stdio.h> //printf ve scanf için gereklidir
int main() {
    float kapasiteOrani;
    printf("Kapasite Oranını (0.00-1.00) Giriniz:");
    scanf("%f",&kapasiteOrani); //önündeki adres(&) işlecine dikkat edelim!
    printf("Girilen Kapasite Oranı: %.2f\n",kapasiteOrani);
    printf("%% olarak Girilen Kapasite Oranı: %%2.2f", 100*kapasiteOrani);
    return 0;
}
/*Program Çıktısı:
Kapasite Oranını (0.00-1.00) Giriniz:0.75
Girilen Kapasite Oranı: 0.75
% olarak Girilen Kapasite Oranı: %75.00
*/
```

Benzer şekilde yarıçapı tamsayı olarak klavyeden okuyan ve çemberin çevresini hesaplayan program aşağıda verilmiştir;

```
/* Bu program, detaylı biçimlendirme scanf örneğidir. */
#include <stdio.h>
int main() {
    int yaricap;
    float cevre;
    printf("Çemberin yarıçapını tamsayı olarak giriniz:");
    scanf("%d",&yaricap);
    cevre=2*3.1415*yaricap;
    printf("Yarıçapı %d olan çemberin çevresi: %.2f",yaricap,cevre);
    return 0;
}
/*Program Çıktısı:
Çemberin yarıçapını tamsayı olarak giriniz:4
Yarıçapı 4 olan çemberin çevresi: 25.13
*/
```

5.4 Düşün ve Çöz

- ◊ **Düşün:** **printf()** ve **scanf()** fonksiyonlarındaki biçim belirleyicilerin (**%d**, **%f** vb.) bellekteki verinin yorumlanmasıdaki önemini açıklayınız.
- ◊ **Düşün:** **scanf()** fonksiyonu ile boşluk içeren bir metni (örneğin ad-soyad) okumaya çalışırken neden sadece ilk kelime alınır? Çözümü nedir?
- ◊ **Düşün:** Standart hata akışı (**stderr**) ile standart çıktı akışı (**stdout**) arasındaki fark nedir? Hataları neden ayrı bir akıştan göndeririz?

- ◇ **Çöz:** `printf()` kullanarak ekrana isim, okul numarası ve not sütunlarından oluşan, düzgün hizalanmış bir tablo bastıran kodu yazınız.
- ◇ **Çöz:** Kullanıcıdan bir dairenin yarıçapını alıp, alanı ve çevresini ekrana virgülden sonra iki basamak gösterecek şekilde yazdıran programı kodlayınız.
- ◇ **Çöz:** Kaçış dizileri (**escape sequences**) kullanarak ekrana tam olarak şu metni yazdırınız: `C:\Program Files\Kitap\C_Dili`